

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических
систем и электроники

**ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
дисциплины
«Искусственный интеллект и машинное обучение»**

Выполнил:
Литвинов Андрей Дмитриевич
2 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника», очная
форма обучения

(подпись)

Работу проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники Института перспективной
инженерии
Воронкин Роман Александрович

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2024 г.

Тема: Работа с Jupiter Notebook, JupiterLab и Google Colab.

Цель работы: исследовать базовые возможности интерактивных оболочек Jupiter Notebook, JupiterLab и Google Colab для языка программирования Python.

Ход выполнения работы

Ссылка на репозиторий GitHub: https://github.com/andlitt/AI_labs_ncfu

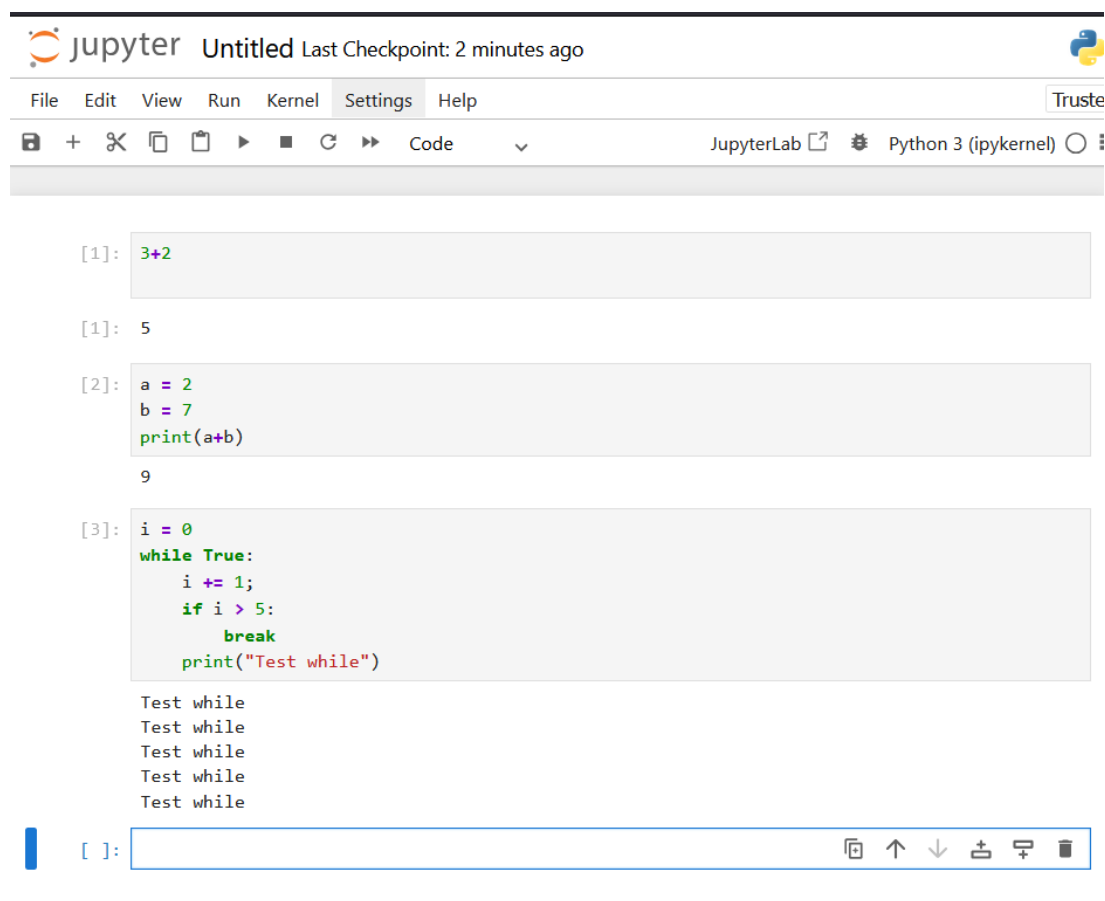


Рисунок 1 – Ввод команд в Jupiter notebook

```
[5]: from matplotlib import pylab as plt
      %matplotlib inline
```

```
[8]: x = [i for i in range(50)]
      y = [i**2 for i in range(50)]
      plt.plot(x,y)
```

```
[8]: [<matplotlib.lines.Line2D at 0x1a2ffb8d8e0>]
```

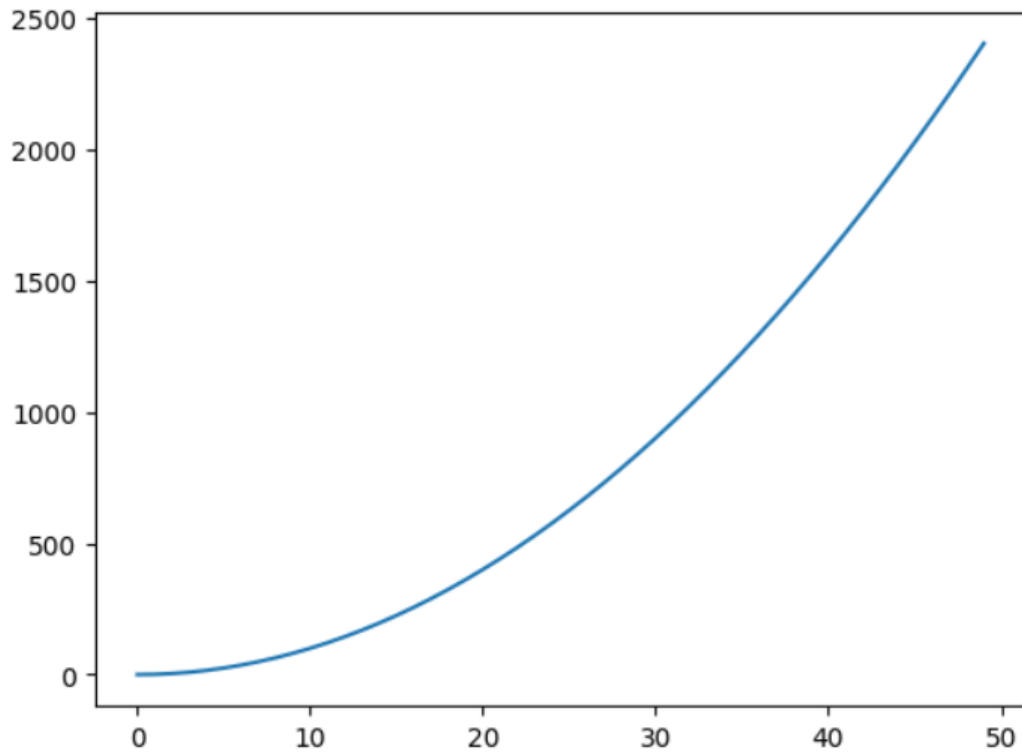


Рисунок 2 – построение графика в Jupiter notebook

```
[9]: %lsmagic
```

```
[9]: ▾ root
      ▶ line
      ▶ cell
```

Рисунок 3 – команда %lsmagic

```
[10]: %env Test = 5
```

```
env: Test=5
```

Рисунок 4 – команда %env

```
[12]: %%time
import time
for i in range(50):
    time.sleep(0.1)

CPU times: total: 0 ns
Wall time: 5.03 s
```

Рисунок 5 – команда %%time

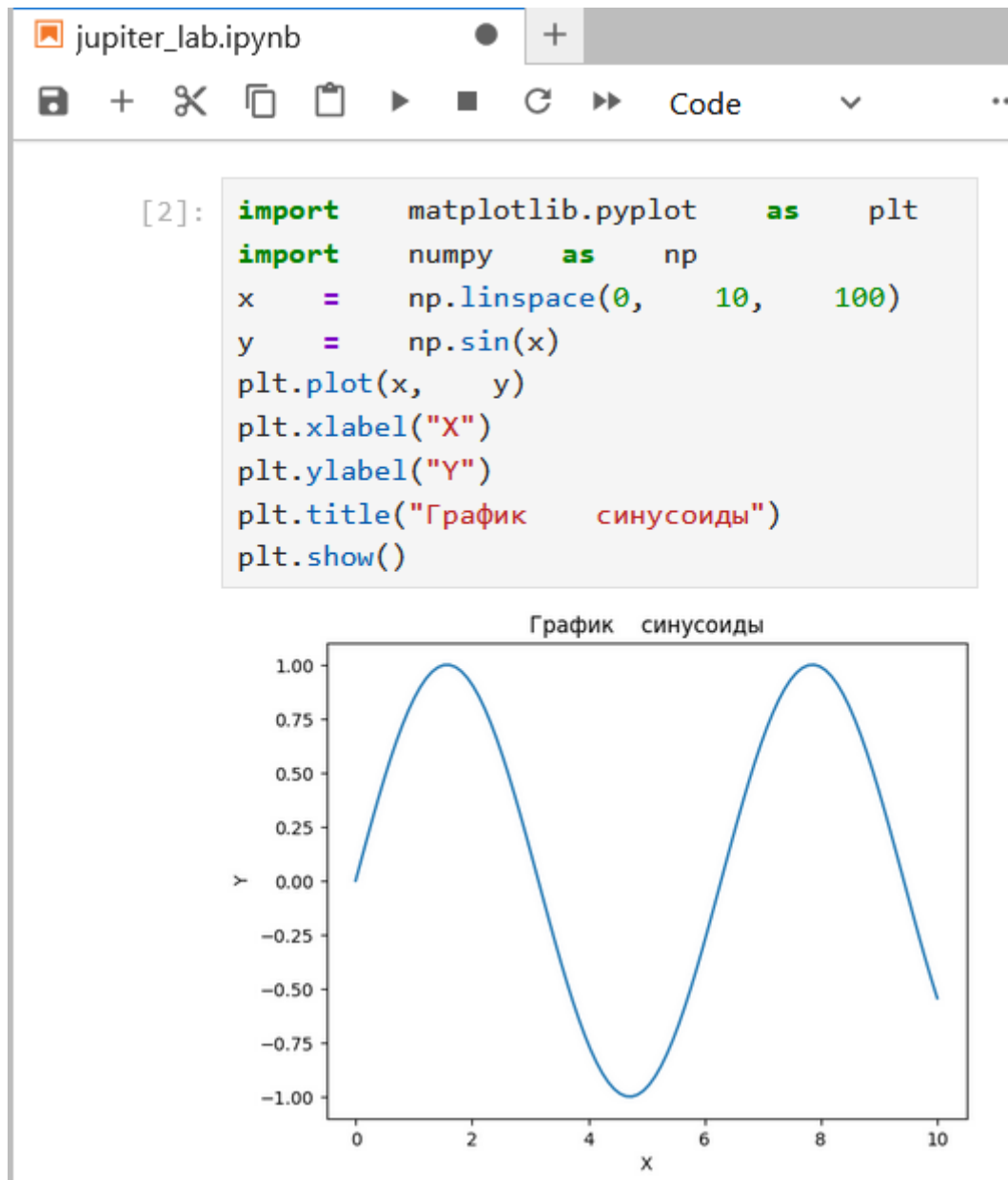


Рисунок 6 – построение графика в Jupyter Lab

Работа с ячейками и Markdown:

Цель: освоить работу с различными типами ячеек (код, Markdown) и форматированием текста.

1. Создайте новую Markdown-ячейку и:

- Напишите заголовок "Практическое задание №1".
- Добавьте жирный и курсивный текст.
- Создайте нумерованный и маркированный списки.
- Вставьте согласно индивидуального задания:

4. Интеграл от функции косинуса:

$$\int \cos(x) dx = \sin(x) + C.$$

Вставьте изображение через ![Описание](URL) .

2. Создайте ячейку Python-кода и:

- Запросите у пользователя его имя с помощью `input()` .
- Выведите приветствие: "Привет, <имя>! Добро пожаловать в JupyterLab / Google Colab!" .
- Запустите ячейку (Shift + Enter).



Рисунок 7 – Задание №1

Работа с файлами

Цель: изучить загрузку, создание и сохранение файлов в JupyterLab / Google

Задания:

1. Создайте и сохраните текстовый файл с помощью `open()` .
2. Запишите в него несколько строк текста.
3. Закройте файл и затем откройте его снова, считав содержимое и выведя на экран.
4. Проверьте, существует ли файл, используя `os.path.exists()` .
5. Удалите файл с помощью модуля `os` .

Работа с файлами

```
import os # Импортируем модуль os

with open('example.txt', 'w') as f:
    f.write('1. Создайте и сохраните текстовый файл с помощью open()\n')
    f.write('2. Запишите в него несколько строк текста.\n')
    f.write('3. Закройте файл и затем откройте его снова, считав содержимое и выведя на экран.\n')
    f.write('4. Проверьте, существует ли файл, используя os.path.exists().\n')
    f.write('5. Удалите файл с помощью модуля os.\n')

with open('example.txt', 'r') as f:
    content = f.read()
    print("Содержимое файла:\n", content)

print("Файл существует:", os.path.exists("example.txt"))

os.remove("example.txt")
print("Файл удален.")

print("Файл существует после удаления:", os.path.exists("example.txt"))
```

Содержимое файла:

1. Создайте и сохраните текстовый файл с помощью open()
2. Запишите в него несколько строк текста.
3. Закройте файл и затем откройте его снова, считав содержимое и выведя на экран.
4. Проверьте, существует ли файл, используя os.path.exists().
5. Удалите файл с помощью модуля os.

Файл существует: True
Файл удален.
Файл существует после удаления: False

Рисунок 8 – Задание №2

Магические команды Jupiter

Цель: изучить магические команды для удобной работы в JupyterLab / Google

Задания:

1. Выведите список всех доступных магических команд (`%lsmagic`).
2. Используйте `%time` и `%%timeit` для измерения времени выполнения кода.
3. Создайте Python-скрипт в Jupyter (`%%writefile script.py`) и выполните его через `!python script.py`.
4. Выведите список файлов в текущей директории с помощью `%ls`.
5. Используйте `%history` для просмотра истории команд.

```
Магические команды Jupiter

[39] %lsmagic
Показать скрытые выходные данные

# Запись кода в файл
%%writefile script.py
for i in range(3):
    print("Итерация {i}")
# Выполнение скрипта
!python test_script.py
Overwriting script.py

[41] # Измерение времени выполнения
%time sum(range(1000000))

%timeit
total = 0
for i in range(1000000):
    total += i

%history

# Проверяем, что файл создан
!cat example.txt

with open('example.txt', 'w') as f:
    f.write('1. Создайте и сохраните текстовый файл с помощью open()\n')
    f.write('2. Запишите в него несколько строк текста.\n')
    f.write('3. Закройте файл и затем откройте его снова, считав содержимое и выведя на экран.\n')
    f.write('4. Проверьте, существует ли файл, используя os.path.exists() .\n')
    f.write('5. Удалите файл с помощью модуля os .\n')

with open('example.txt', 'r') as f:
    content = f.read()
    print("Содержимое файла:\n", content)

print("Файл существует:", os.path.exists("example.txt"))
os.remove("example.txt")
name = input()
print(f"Привет, {name}! Добро пожаловать в Google Colab!")
name = input()
```

Рисунок 9 – Задание №3

Взаимодействие с оболочкой системы

Цель: изучить выполнение команд терминала прямо из JupyterLab / Google Colab.

Задания:

1. Выведите список файлов в текущей директории с помощью `!ls` .
2. Проверьте, какой Python используется (`!which python`).
3. Создайте папку `test_folder` (`!mkdir test_folder`) и убедитесь, что она

появилась.

4. Переместите файл в новую папку и удалите его.
5. Очистите вывод в ячейке (`!clear`).

Взаимодействие с оболочкой системы

```
print("Список файлов в текущей директории:")
!ls

print("\nПуть к интерпретатору Python:")
!which python

print("\nСоздание папки test_folder:")
!mkdir test_folder
print("Содержимое текущей директории после создания папки:")
!ls

print("\nСоздание файла example.txt:")
with open('example.txt', 'w') as f:
    f.write('Это тестовый файл.')
!ls

print("\nПеремещение файла example.txt в папку test_folder:")
!mv example.txt test_folder/
print("Содержимое папки test_folder:")
!ls test_folder

print("\nУдаление файла example.txt из папки test_folder:")
!rm test_folder/example.txt
print("Содержимое папки test_folder после удаления файла:")
!ls test_folder

print("\nОчистка вывода в ячейке:")
!clear
```

Список файлов в текущей директории:
sample_data script.py test_folder

Путь к интерпретатору Python:
/usr/local/bin/python

Создание папки test_folder:
mkdir: cannot create directory 'test_folder': File exists

Содержимое текущей директории после создания папки:
sample_data script.py test_folder

Рисунок 10 – Задание №4

Работа с Google Drive в Google Collab

Цель: научиться подключать Google Drive, загружать и сохранять файлы, работать с обычными текстовыми файлами.

1. Подключите Google Drive к Colab:

- Проверьте, что ваш диск успешно подключился, используя `!ls /content/drive/MyDrive` .

2. Создайте и сохраните текстовый файл в Google Drive:

- Откройте файл `my_text_file.txt` в режиме записи.
- Запишите в него несколько строк текста.
- Закройте файл и убедитесь, что он появился в папке `MyDrive`

3. Прочитайте файл из Google Drive:

- Откройте ранее созданный файл.
- Считайте его содержимое и выведите в ячейке.

4. Создайте и сохраните CSV-файл вручную, используя Microsoft Excel или Libre Office Calc:

- Создайте список, в котором будут строки с данными о студентах (например, ФИО, возраст, группа).
- Запишите этот список в CSV-файл в Google Drive вручную, используя стандартные методы записи в файл.

Работа с Google Drive в Google Colab

```
[46] from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
from google.colab import drive

# Подключение Google Drive
drive.mount('/content/drive')
# Запись текста в файл
file_path = "/content/drive/MyDrive/my_text_file.txt"

with open(file_path, "w") as f:
    f.write("Это тестовый файл, сохраненный в Google Drive.\n")
    f.write("Вторая строка текста.")
print("Файл успешно сохранен в Google Drive.")
# Чтение файла
with open(file_path, "r") as f:
    content = f.read()
    print("Содержимое файла:\n", content)

# Создание CSV вручную
students = [
    ["Иванов И.И.", 20, "ИВТ-101"],
    ["Петров П.П.", 22, "ИВТ-102"],
    ["Сидорова А.А.", 21, "ИВТ-103"]
]

csv_path = "/content/drive/MyDrive/students.csv"
with open(csv_path, "w") as f:
    for student in students:
        f.write(",".join(map(str, student)) + "\n")
print("Файл students.csv успешно сохранен в Google Drive.")
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Файл успешно сохранен в Google Drive.

Содержимое файла:
Это тестовый файл, сохраненный в Google Drive.
Вторая строка текста.

Файл students.csv успешно сохранен в Google Drive.

Рисунок 11 – Текст задания №4

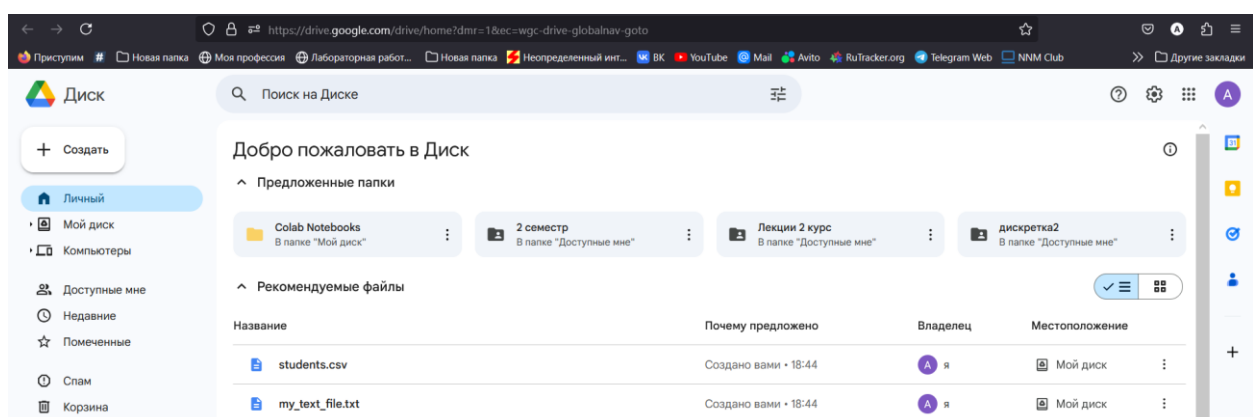


Рисунок 12 – Созданные файлы на Google Drive

1. Основные отличия JupyterLab от Jupyter Notebook:

- **Интерфейс:** JupyterLab предоставляет более современный и гибкий интерфейс с поддержкой вкладок, окон и панелей, что позволяет работать с

несколькими файлами и инструментами одновременно. Jupyter Notebook имеет более простой интерфейс с одним ноутбуком на экране.

- **Функциональность:** JupyterLab поддерживает расширенные функции, такие как встроенный терминал, текстовый редактор, просмотрщик файлов и интеграция с другими инструментами (например, отладчик). Jupyter Notebook ограничен работой с ноутбуками.
- **Модульность:** JupyterLab позволяет настраивать рабочую среду, добавляя или удаляя панели и инструменты.

2. Как создать новую рабочую среду (ноутбук) в JupyterLab:

1. Откройте JupyterLab.
2. В левой панели выберите **Launcher** (запускается автоматически при старте).
3. В разделе **Notebook** выберите ядро (например, Python 3).
4. Новый ноутбук откроется в новой вкладке.

3. Типы ячеек в JupyterLab и их переключение:

- **Code:** Ячейка для написания и выполнения кода.
- **Markdown:** Ячейка для написания текста в формате Markdown.
- **Raw:** Ячейка для неформатированного текста.

Переключение типов ячеек:

- Используйте выпадающее меню в верхней панели ноутбука.
- Или используйте горячие клавиши:
 - Esc + Y — переключить на Code.
 - Esc + M — переключить на Markdown.

- Esc + R — переключить на Raw.

4. Как выполнить код в ячейке и горячие клавиши:

- Нажмите Shift + Enter для выполнения кода и перехода к следующей ячейке.
- Нажмите Ctrl + Enter для выполнения кода без перехода к следующей ячейке.
- Нажмите Alt + Enter для выполнения кода и создания новой ячейки ниже.

5. Как запустить терминал или текстовый редактор в JupyterLab:

- **Терминал:** В Launcher выберите **Terminal**.
- **Текстовый редактор:** В Launcher выберите **Text File** или откройте файл с расширением .txt, .py и т.д.

6. Инструменты для работы с файлами и каталогами:

- **Файловый менеджер:** В левой панели JupyterLab отображается структура каталогов.
- **Перетаскивание:** Файлы можно перетаскивать между панелями.
- **Контекстное меню:** Правый клик на файле или папке для копирования, удаления, переименования и других действий.

7. Управление ядрами (kernels) в JupyterLab:

- **Перезапуск ядра:** В меню **Kernel** выберите **Restart Kernel**.
- **Смена ядра:** В меню **Kernel** выберите **Change Kernel** и выберите нужное ядро.

- **Остановка ядра:** В меню **Kernel** выберите **Shutdown Kernel**.

8. Возможности системы вкладок и окон:

- **Вкладки:** Можно открывать несколько файлов и инструментов в одной вкладке.
- **Разделение окон:** Панели можно разделять горизонтально или вертикально для одновременной работы с несколькими файлами.
- **Перетаскивание:** Вкладки и панели можно перетаскивать для создания новых окон.

9. Магические команды для измерения времени выполнения кода:

- `%%time`: Измеряет время выполнения всей ячейки.
- `%time`: Измеряет время выполнения одной строки кода.
- `%%timeit`: Запускает код несколько раз и выводит среднее время выполнения.

10. Магические команды для запуска кода на других языках:

- `%%bash`: Запуск команд Bash.
- `%%html`: Вставка HTML-кода.
- `%%javascript`: Запуск JavaScript.

11. Основные отличия Google Colab от JupyterLab:

- **Облачная среда:** Google Colab работает в облаке, а JupyterLab — локально или на сервере.
- **Доступ к GPU/TPU:** Colab предоставляет бесплатный доступ к GPU и TPU.
- **Интеграция с Google Drive:** Colab позволяет легко работать с файлами из Google Drive.

- **Ограничения:** Colab имеет ограничения на время выполнения и ресурсы для бесплатных пользователей.

12. Как создать новый ноутбук в Google Colab:

1. Откройте [Google Colab](#).
2. Нажмите **Новый блокнот** в разделе **Файл** → **Создать блокнот**.

13. Типы ячеек в Google Colab и их переключение:

- **Code:** Ячейка для написания и выполнения кода.
- **Text:** Ячейка для написания текста (Markdown).

Переключение:

- Используйте выпадающее меню в верхней части ячейки.
- Или используйте горячие клавиши:
 - Ctrl + M + Y — переключить на Code.
 - Ctrl + M + M — переключить на Text.

14. Как выполнить код в ячейке Google Colab и горячие клавиши:

- Нажмите Shift + Enter для выполнения кода и перехода к следующей ячейке.
- Нажмите Ctrl + Enter для выполнения кода без перехода к следующей ячейке.

15. Способы загрузки и сохранения файлов в Google Colab:

- **Загрузка:** Используйте `files.upload()` для загрузки файлов с локального компьютера.
- **Сохранение:** Используйте `files.download()` для скачивания файлов.

16. Подключение Google Drive к Google Colab:

```
from google.colab import drive  
drive.mount('/content/drive')
```

После этого файлы из Google Drive будут доступны в папке `/content/drive/My Drive`.

17. Загрузка файлов из локального компьютера:

```
from google.colab import files  
uploaded = files.upload()
```

18. Просмотр списка файлов в Google Colab:

```
!ls
```

Или используйте файловый менеджер в левой панели.

19. Магические команды для измерения времени выполнения кода в Google Colab:

- `%%time`: Измеряет время выполнения всей ячейки.

- `%time`: Измеряет время выполнения одной строки кода.
- `%%timeit`: Запускает код несколько раз и выводит среднее время

выполнения.

Пример:

```
%%time  
for i in range(1000000):  
    pass
```

20. Изменение аппаратных ресурсов в Google Colab:

1. Перейдите в **Среда выполнения** → **Изменить тип среды выполнения**.
2. Выберите **Аппаратный ускоритель** (None, GPU, TPU).
3. Нажмите **Сохранить**.

После этого Colab перезапустится с выбранными ресурсами.

Вывод: В ходе выполнения работы были исследованы базовые возможности интерактивных оболочек Jupyter Notebook, JupyterLab и Google Colab для языка программирования Python.